GUIA COMPLETO DE

Vulnerabilidades Web
&
Defesas Escaláveis para SaaS

Documento de referência para equipes de desenvolvimento, DevOps e IA

Versão 1.0 — Fevereiro 2026

Sumário

1. Introdução e Objetivo3
2. OWASP Top 10 — Visão Geral e Defesas4
3. Injection (SQL, NoSQL, Command, LDAP)5
4. Broken Authentication & Session Management7
5. Cross-Site Scripting (XSS)9
6. Insecure Direct Object References (IDOR/BOLA)11
7. Security Misconfiguration12
8. Sensitive Data Exposure14
9. Cross-Site Request Forgery (CSRF)15
10. Server-Side Request Forgery (SSRF)16
11. Broken Access Control17
12. Vulnerable Dependencies18
13. Rate Limiting & DDoS Protection19
14. Upload de Arquivos20
15. APIs — Segurança Específica21
16. Infraestrutura & DevSecOps22
17. Monitoramento, Logging & Incident Response24
18. Checklist Final de Segurança25
19. Referências e Recursos27

1. Introdução e Objetivo

Este documento é um guia prático e abrangente sobre todas as falhas e vulnerabilidades conhecidas da web moderna. Ele foi projetado para ser consultado por desenvolvedores, equipes de segurança e, especialmente, por agentes de IA que auxiliam na construção e auditoria de software.

Cada vulnerabilidade é apresentada com: o que é, como um atacante a explora, exemplos práticos de código vulnerável, a correção recomendada com código seguro, e medidas de defesa em profundidade para proteger serviços SaaS de forma escalável.

Princípios Fundamentais de Segurança

Defense in Depth: Nunca dependa de uma única camada de proteção. Combine validação no frontend, backend, banco de dados e infraestrutura.

Least Privilege: Cada componente deve ter apenas as permissões mínimas necessárias para funcionar.

Fail Secure: Quando algo falha, o sistema deve negar acesso por padrão, não conceder.

Zero Trust: Nunca confie em dados que vêm do cliente. Valide tudo no servidor.

2. OWASP Top 10 — Visão Geral e Defesas

A OWASP (Open Worldwide Application Security Project) publica periodicamente a lista das 10 vulnerabilidades mais críticas em aplicações web. A tabela abaixo apresenta um resumo atualizado com base no OWASP Top 10:2021 e nas tendências de 2024-2025.

Posição	Vulnerabilidade	Defesa Principal
A01	Broken Access Control	RBAC + verificação server-side em cada endpoint
A02	Cryptographic Failures	TLS 1.3, AES-256, hashing com bcrypt/Argon2
A03	Injection	Prepared statements, ORM, input validation
A04	Insecure Design	Threat modeling, security requirements desde o design
A05	Security Misconfiguration	Hardening automatizado, IaC seguro, headers HTTP
A06	Vulnerable Components	Dependabot, npm audit, SBOM, lock files
A07	Auth & Identification Failures	MFA, session management, rate limiting
A08	Data Integrity Failures	Verificação de integridade, CI/CD seguro, SRI
A09	Logging & Monitoring Failures	SIEM, alertas, audit trail imutável
A10	Server-Side Request Forgery	Allowlist de URLs, bloqueio de IPs internos

3. Injection (SQL, NoSQL, Command, LDAP)

Injection é a inserção de código malicioso em queries ou comandos interpretados pelo servidor. Continua sendo uma das vulnerabilidades mais destrutivas, permitindo roubo de dados, bypass de autenticação e até controle total do servidor.

3.1 SQL Injection

O que é: O atacante injeta código SQL através de inputs do usuário que são concatenados diretamente em queries.

Código VULNERÁVEL (Node.js + MySQL):

```
const query = `SELECT * FROM users WHERE email = '${req.body.email}' AND password =
'${req.body.password}'`;
db.query(query); // PERIGO: SQL Injection
```

Ataque: O atacante envia no campo email: ' OR '1'='1' -- e obtém acesso a todos os registros.

Código SEGURO (Prepared Statements):

```
const query = "SELECT * FROM users WHERE email = ? AND password = ?";
db.query(query, [req.body.email, hashedPassword]);
```

Código SEGURO (ORM — Prisma):

```
const user = await prisma.user.findUnique({ where: { email: req.body.email } });
```

3.2 NoSQL Injection (MongoDB)

Código VULNERÁVEL:

```
db.users.find({ email: req.body.email, password: req.body.password });
```

Se o atacante enviar { "$gt": "" } como password, o MongoDB retorna todos os documentos.

Código SEGURO:

```
// Validar tipo dos inputs
if (typeof req.body.email !== "string" || typeof req.body.password !== "string") {
  return res.status(400).json({ error: "Invalid input" });
}
// Usar sanitização com mongo-sanitize
const sanitize = require("mongo-sanitize");
db.users.find({ email: sanitize(req.body.email) });
```

3.3 Command Injection

Código VULNERÁVEL:

```
const { exec } = require("child_process");
exec(`ping ${req.query.host}`); // PERIGO
```

Ataque: host=google.com; rm -rf / — executa comandos arbitrários no servidor.

Código SEGURO:

```
const { execFile } = require("child_process");
// execFile não usa shell, impede injection
execFile("ping", ["-c", "4", sanitizedHost]);
```

3.4 Defesas em Profundidade para Injection

Camada	Ação
Código	Prepared statements, ORMs, validação de tipo/schema (Zod, Joi)
WAF	Web Application Firewall (Cloudflare WAF, AWS WAF) com regras anti-SQLi
Banco de Dados	Usuário do DB com permissões mínimas (SELECT only para leitura)
Monitoramento	Logs de queries suspeitas, alertas para patterns de injection
Infraestrutura	Não rodar aplicação como root, sandboxing de processos

4. Broken Authentication & Session Management

Falhas de autenticação permitem que atacantes assumam identidades de outros usuários, acessem contas administrativas ou sequestrem sessões ativas.

4.1 Vulnerabilidades Comuns

- Senhas fracas permitidas sem validação de complexidade
- Brute force sem rate limiting — atacante tenta milhares de combinações
- Tokens de sessão previsíveis ou que não expiram
- Armazenamento de senhas em texto puro ou com hash fraco (MD5, SHA1)
- Recuperação de senha insegura (perguntas secretas, links sem expiração)
- Session fixation — sessão não é regenerada após login

4.2 Hashing Seguro de Senhas

NUNCA USE: MD5, SHA1, SHA256 puro para senhas.

USE: bcrypt (cost factor >= 12) ou Argon2id.

Código SEGURO (bcrypt):

```
const bcrypt = require("bcrypt");
const SALT_ROUNDS = 12;

// Ao registrar
const hash = await bcrypt.hash(password, SALT_ROUNDS);
await db.user.create({ data: { email, password: hash } });

// Ao fazer login
const valid = await bcrypt.compare(inputPassword, user.password);
```

4.3 JWT — Boas Práticas

Código VULNERÁVEL:

```
// Token sem expiração, secret fraco
jwt.sign({ userId: user.id }, "secret123");
```

Código SEGURO:

```
const token = jwt.sign(
  { sub: user.id, role: user.role },
  process.env.JWT_SECRET, // Mínimo 256 bits de entropia
```

```
  { expiresIn: "15m", algorithm: "HS256" }
);

// Refresh token: longa duração, armazenado em httpOnly cookie
const refreshToken = jwt.sign({ sub: user.id }, process.env.REFRESH_SECRET, {
expiresIn: "7d" });
res.cookie("refreshToken", refreshToken, {
  httpOnly: true, secure: true, sameSite: "strict", maxAge: 7 * 24 * 60 * 60 * 1000
});
```

4.4 Rate Limiting para Login

```
const rateLimit = require("express-rate-limit");

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutos
  max: 5, // 5 tentativas
  message: { error: "Muitas tentativas. Tente novamente em 15 min." },
  standardHeaders: true,
  keyGenerator: (req) => req.body.email || req.ip, // Limita por email
});

app.post("/api/auth/login", loginLimiter, loginHandler);
```

4.5 Checklist de Autenticação Segura

Item	Implementação
Hashing de senha	bcrypt (cost 12+) ou Argon2id
MFA	TOTP (Google Authenticator) ou WebAuthn/Passkeys
Rate limiting	5 tentativas/15min por email + captcha após 3 falhas
Session	Regenerar ID após login, httpOnly + Secure cookies
JWT	Access token 15min, refresh token 7d, rotação de refresh
Recuperação	Token único com expiração de 1h, invalidar após uso
Logout	Invalidar token no backend (blocklist Redis)

5. Cross-Site Scripting (XSS)

XSS permite que atacantes injetem scripts maliciosos em páginas vistas por outros usuários. É usado para roubo de cookies, redirecionamento para phishing, keylogging e defacement.

5.1 Tipos de XSS

Tipo	Mecanismo	Exemplo
Stored (Persistente)	Script salvo no banco e exibido a outros usuários	Comentário com <script> em um fórum
Reflected	Script vem na URL e é refletido na resposta	Link malicioso com payload na query string
DOM-based	Script manipula o DOM diretamente no client	innerHTML com dados não sanitizados da URL

5.2 Exemplos Práticos

Código VULNERÁVEL (React com dangerouslySetInnerHTML):

```
// PERIGO: renderiza HTML do usuário sem sanitização
function Comment({ text }) {
  return <div dangerouslySetInnerHTML={{ __html: text }} />;
}
```

Código SEGURO:

```
import DOMPurify from "dompurify";


function Comment({ text }) {
  // Opção 1: Sanitizar HTML
  const clean = DOMPurify.sanitize(text, { ALLOWED_TAGS: ["b", "i", "em", "strong",
"a"] });
  return <div dangerouslySetInnerHTML={{ __html: clean }} />;


  // Opção 2 (melhor): Não usar HTML, usar texto puro
  return <div>{text}</div>; // React escapa automaticamente
}
```

Código VULNERÁVEL (Server-side rendering):

```
app.get("/search", (req, res) => {
  res.send(`<h1>Resultados para: ${req.query.q}</h1>`); // XSS Reflected
});
```

Código SEGURO:

```
const escapeHtml = require("escape-html");
app.get("/search", (req, res) => {
  res.send(`<h1>Resultados para: ${escapeHtml(req.query.q)}</h1>`);
});
```

5.3 Content Security Policy (CSP)

CSP é a defesa mais poderosa contra XSS. Ela define de onde scripts podem ser carregados.

Header HTTP recomendado:

```
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'
'unsafe-inline'; img-src 'self' data: https:; font-src 'self'; connect-src 'self'
https://api.seudominio.com; frame-ancestors 'none'; base-uri 'self'; form-action
'self';
```

Implementação em Express:

```
const helmet = require("helmet");
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      imgSrc: ["'self'", "data:", "https:"],
      connectSrc: ["'self'", "https://api.seudominio.com"],
      frameAncestors: ["'none'"],
    },
  },
}));
```

6. Insecure Direct Object References (IDOR/BOLA)

IDOR (ou BOLA — Broken Object Level Authorization) ocorre quando o sistema expõe referências internas a objetos (IDs de banco) sem verificar se o usuário tem permissão para acessá-los.

Código VULNERÁVEL:

```
// GET /api/invoices/123
app.get("/api/invoices/:id", async (req, res) => {
  const invoice = await db.invoice.findUnique({ where: { id: req.params.id } });
  res.json(invoice); // Qualquer usuário acessa qualquer fatura!
});
```

Código SEGURO:

```
app.get("/api/invoices/:id", authenticate, async (req, res) => {
  const invoice = await db.invoice.findFirst({
    where: {
      id: req.params.id,
      organizationId: req.user.organizationId, // Filtra pelo tenant
    },
  });
  if (!invoice) return res.status(404).json({ error: "Not found" });
  res.json(invoice);
});
```

6.1 Estratégias Escaláveis

- **UUIDs em vez de IDs sequenciais:** Não elimina IDOR, mas dificulta enumeração.
- **Middleware de autorização:** Crie um middleware que automaticamente filtra por tenant/organization.
- **Row Level Security (RLS):** Use RLS no PostgreSQL para garantir isolamento no nível do banco.

Exemplo de RLS no PostgreSQL:

```
ALTER TABLE invoices ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON invoices
  USING (organization_id = current_setting('app.current_org')::uuid);
```

7. Security Misconfiguration

Misconfiguration é quando servidores, frameworks ou serviços em nuvem são deixados com configurações padrão inseguras. É a vulnerabilidade mais comum e frequentemente a mais fácil de explorar.

7.1 Headers de Segurança Obrigatórios

Header	Valor Recomendado
Strict-Transport-Security	max-age=63072000; includeSubDomains; preload
X-Content-Type-Options	nosniff
X-Frame-Options	DENY
Referrer-Policy	strict-origin-when-cross-origin
Permissions-Policy	camera=(), microphone=(), geolocation=()
Content-Security-Policy	Conforme seção 5.3
X-XSS-Protection	0 (desabilite — CSP é superior)
Cache-Control	no-store para dados sensíveis

Implementação completa com Helmet.js:

```
const helmet = require("helmet");
app.use(helmet());
app.use(helmet.hsts({ maxAge: 63072000, includeSubDomains: true, preload: true }));
app.use(helmet.referrerPolicy({ policy: "strict-origin-when-cross-origin" }));
app.use(helmet.permittedCrossDomainPolicies());
```

7.2 Configurações Críticas

- Remover headers que revelam tecnologia: X-Powered-By, Server
- Desabilitar listagem de diretórios no servidor web
- Não expor stack traces em produção (NODE_ENV=production)
- Proteger ou remover endpoints de debug e healthcheck detalhado
- Desabilitar métodos HTTP desnecessários (TRACE, OPTIONS sem CORS)
- Bloquear acesso a arquivos sensíveis (.env, .git, docker-compose.yml)

7.3 CORS — Configuração Segura

Código VULNERÁVEL:

```
app.use(cors({ origin: "*" })); // Permite qualquer domínio!
```

Código SEGURO:

```
const allowedOrigins = ["https://app.seudominio.com",
"https://admin.seudominio.com"];
app.use(cors({
  origin: (origin, callback) => {
    if (!origin || allowedOrigins.includes(origin)) callback(null, true);
    else callback(new Error("CORS not allowed"));
  },
  credentials: true,
  methods: ["GET", "POST", "PUT", "DELETE"],
  allowedHeaders: ["Content-Type", "Authorization"],
}));
```

8. Sensitive Data Exposure

Dados sensíveis expostos através de transmissão insegura, armazenamento inadequado ou respostas de API que retornam mais informação do que o necessário.

8.1 Criptografia em Trânsito

- TLS 1.3 obrigatório — desabilitar TLS 1.0 e 1.1
- HSTS com preload para forçar HTTPS
- Certificados via Let's Encrypt com renovação automática
- Pinning de certificado para apps mobile (opcional mas recomendado)

8.2 Criptografia em Repouso

- Banco de dados: encryption at rest (AWS RDS, GCP Cloud SQL habilitam por padrão)
- Dados sensíveis do usuário: criptografia em nível de campo com AES-256-GCM
- Secrets: usar serviço de gerenciamento (AWS Secrets Manager, Vault)
- Nunca armazenar secrets em código ou variáveis de ambiente não protegidas

Criptografia de campo sensível:

```
const crypto = require("crypto");
const ALGORITHM = "aes-256-gcm";
const KEY = Buffer.from(process.env.ENCRYPTION_KEY, "hex"); // 32 bytes

function encrypt(text) {
  const iv = crypto.randomBytes(16);
  const cipher = crypto.createCipheriv(ALGORITHM, KEY, iv);
  let encrypted = cipher.update(text, "utf8", "hex");
  encrypted += cipher.final("hex");
  const tag = cipher.getAuthTag().toString("hex");
  return `${iv.toString("hex")}:${tag}:${encrypted}`;
}
```

8.3 Respostas de API — Princípio do Mínimo Necessário

VULNERÁVEL — retorna tudo:

```
res.json(user); // Expõe password hash, tokens internos, etc.
```

SEGURO — projection explícita:

```
const { id, name, email, avatar } = user;
res.json({ id, name, email, avatar }); // Só o necessário
```

9. Cross-Site Request Forgery (CSRF)

CSRF força o navegador do usuário autenticado a enviar requests maliciosos para sua aplicação, usando os cookies de sessão existentes.

9.1 Defesas

1. SameSite Cookies (defesa principal):

```
res.cookie("session", token, {
  httpOnly: true,
  secure: true,
  sameSite: "strict", // Bloqueia envio cross-origin
  maxAge: 3600000,
});
```

2. CSRF Token (para forms tradicionais):

```
const csrf = require("csurf");
app.use(csrf({ cookie: { httpOnly: true, secure: true, sameSite: "strict" } }));

// No template
app.get("/form", (req, res) => {
  res.render("form", { csrfToken: req.csrfToken() });
});
```

3. Para APIs SPA (React/Vue/Next.js):

Se a autenticação usa Bearer tokens no header Authorization (não cookies), CSRF não se aplica. Se usa cookies, implemente o Double Submit Cookie pattern ou use SameSite=Strict.

10. Server-Side Request Forgery (SSRF)

SSRF ocorre quando o servidor faz requests para URLs controladas pelo atacante, permitindo acesso a serviços internos, metadata de cloud e rede privada.

Código VULNERÁVEL:

```
// Endpoint que busca conteúdo de uma URL
app.post("/api/fetch-url", async (req, res) => {
  const response = await fetch(req.body.url); // PERIGO
  res.json(await response.json());
});
```

Ataque: O atacante envia url: "http://169.254.169.254/latest/meta-data/" e obtém credenciais AWS.

Código SEGURO:

```
const { URL } = require("url");
const dns = require("dns").promises;
const ipaddr = require("ipaddr.js");

async function isSafeUrl(input) {
  const url = new URL(input);
  if (!["http:", "https:"].includes(url.protocol)) return false;

  const { address } = await dns.lookup(url.hostname);
  const addr = ipaddr.parse(address);
  const range = addr.range();

  const blocked = ["private", "loopback", "linkLocal", "uniqueLocal"];
  if (blocked.includes(range)) return false;

  return true;
}
```

11. Broken Access Control

Broken Access Control é a vulnerabilidade #1 do OWASP Top 10. Inclui IDOR, escalação de privilégio, acesso a endpoints admin sem autorização, e manipulação de parâmetros.

11.1 Middleware de Autorização Escalável

```
// middleware/authorize.js
function authorize(...allowedRoles) {
  return (req, res, next) => {
    if (!req.user) return res.status(401).json({ error: "Não autenticado" });
    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ error: "Sem permissão" });
    }
    next();
  };
}

// Uso
app.delete("/api/users/:id", authenticate, authorize("admin"), deleteUser);
app.get("/api/reports", authenticate, authorize("admin", "manager"), getReports);
```

11.2 Multi-Tenancy Seguro

Para SaaS, cada tenant deve ter isolamento total de dados. A abordagem recomendada em ordem de segurança:

1. Database por tenant (máximo isolamento, mais complexo)
2. Schema por tenant (bom isolamento, moderadamente complexo)
3. Row Level Security com coluna tenant_id (bom para maioria dos SaaS)
4. Filtro na aplicação (mínimo — exige disciplina total)

12. Vulnerable Dependencies

Supply chain attacks e dependências com vulnerabilidades conhecidas são responsáveis por uma porcentagem crescente de breaches. Em 2024, ataques via npm e PyPI packages maliciosos cresceram 300%.

12.1 Proteções

```
# Auditar dependências
npm audit
npm audit --fix

# Usar lockfile
npm ci # Em vez de npm install (respeita o lock)

# GitHub Actions — Dependabot
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 10
```

12.2 Subresource Integrity (SRI)

Para CDNs externas, sempre use SRI para garantir que o script não foi alterado:

```
<script src="https://cdn.example.com/lib.js" integrity="sha384-abc123..."
crossorigin="anonymous"></script>
```

13. Rate Limiting & DDoS Protection

Sem rate limiting, seu SaaS fica vulnerável a brute force, scraping, abuse de API e DDoS na camada de aplicação.

13.1 Estratégia em Camadas

Camada	Ferramenta	Função
Edge/CDN	Cloudflare, AWS CloudFront	DDoS L3/L4, bot protection, WAF
API Gateway	Kong, AWS API Gateway	Rate limiting por API key/IP
Aplicação	express-rate-limit, Redis	Rate limiting granular por rota/usuário
Banco	Connection pooling	Limitar conexões simultâneas

13.2 Rate Limiting Avançado com Redis

```
const { RateLimiterRedis } = require("rate-limiter-flexible");
const Redis = require("ioredis");

const rateLimiter = new RateLimiterRedis({
  storeClient: new Redis(process.env.REDIS_URL),
  keyPrefix: "rl",
  points: 100, // Requests
  duration: 60, // Por minuto
  blockDuration: 300, // Bloqueia 5min se exceder
});

app.use(async (req, res, next) => {
  try {
    const key = req.user?.id || req.ip;
    await rateLimiter.consume(key);
    next();
  } catch (err) {
    res.status(429).json({ error: "Rate limit exceeded" });
  }
});
```

14. Upload de Arquivos

Upload de arquivos é um dos vetores de ataque mais perigosos. Pode levar a execução remota de código, XSS persistente e negação de serviço.

14.1 Regras de Upload Seguro

- Validar MIME type real (magic bytes), não confiar na extensão
- Limitar tamanho máximo (ex: 10MB para imagens, 50MB para documentos)
- Renomear arquivos com UUID — nunca usar o nome original do usuário
- Armazenar fora do webroot (S3, GCS) — nunca no diretório público
- Servir via CDN com Content-Disposition: attachment para downloads
- Escanear com antivírus (ClamAV) para uploads públicos
- Nunca executar/interpretar arquivos uploaded

Implementação segura:

```
const multer = require("multer");
const { v4: uuid } = require("uuid");
const fileType = require("file-type");

const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 10 * 1024 * 1024 }, // 10MB
});

app.post("/api/upload", upload.single("file"), async (req, res) => {
  const type = await fileType.fromBuffer(req.file.buffer);
  const allowed = ["image/jpeg", "image/png", "image/webp", "application/pdf"];

  if (!type || !allowed.includes(type.mime)) {
    return res.status(400).json({ error: "Tipo de arquivo não permitido" });
  }

  const key = `uploads/${uuid()}.${type.ext}`;
  await s3.upload({ Bucket: BUCKET, Key: key, Body: req.file.buffer }).promise();
  res.json({ url: `https://cdn.seudominio.com/${key}` });
});
```

15. APIs — Segurança Específica

APIs são a superfície de ataque principal de qualquer SaaS moderno. As vulnerabilidades de API mais comuns (OWASP API Security Top 10) requerem defesas específicas.

15.1 Validação de Input com Schema

```
const { z } = require("zod");

const CreateUserSchema = z.object({
  name: z.string().min(2).max(100),
  email: z.string().email(),
  password: z.string().min(8).max(128)
    .regex(/[A-Z]/, "Precisa de letra maiúscula")
    .regex(/[0-9]/, "Precisa de número")
    .regex(/[^A-Za-z0-9]/, "Precisa de caractere especial"),
  role: z.enum(["user", "admin"]).default("user"),
});

// Middleware de validação
function validate(schema) {
  return (req, res, next) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      return res.status(400).json({ errors: result.error.flatten() });
    }
    req.validated = result.data;
    next();
  };
}
```

15.2 API Keys & Authentication

- API keys para server-to-server (nunca expor no frontend)
- OAuth 2.0 + PKCE para aplicações de terceiros
- JWT com rotação de refresh tokens para SPAs
- Scoped tokens — cada token tem permissões específicas
- Prefixar API keys para facilitar rotação (ex: sk_live_abc123)

15.3 Versionamento e Deprecação

- Versionar APIs (v1, v2) para não quebrar integrações
- Deprecar versões antigas com aviso de 6 meses

- Manter changelog público de breaking changes

16. Infraestrutura & DevSecOps

16.1 Docker Security

```
# Dockerfile seguro
FROM node:20-alpine

# Não rodar como root
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .

USER appuser
EXPOSE 3000
CMD ["node", "server.js"]
```

- Usar imagens alpine/distroless (menor superfície de ataque)
- Escanear imagens com Trivy ou Snyk Container
- Nunca copiar .env ou secrets para a imagem
- Usar multi-stage builds para não incluir devDependencies

16.2 CI/CD Security

```
# GitHub Actions — Pipeline de segurança
name: Security
on: [push, pull_request]
jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: npm audit
        run: npm audit --audit-level=high
      - name: SAST (Semgrep)
        uses: returntocorp/semgrep-action@v1
        with:
          config: p/owasp-top-ten
      - name: Secret scanning
        uses: trufflesecurity/trufflehog@main
```

```
      - name: Container scan
        run: trivy image myapp:latest
```

16.3 Secrets Management

- NUNCA commitar secrets no repositório
- Usar serviços dedicados: AWS Secrets Manager, HashiCorp Vault, Doppler
- Rotacionar secrets automaticamente (mínimo a cada 90 dias)
- Auditar acesso a secrets (quem acessou, quando)
- Usar .env.example com placeholders, nunca valores reais

17. Monitoramento, Logging & Incident Response

17.1 O que Logar (e o que NÃO logar)

Logar	NUNCA Logar
Tentativas de login (sucesso e falha)	Senhas (nem em texto, nem hash)
Ações de admin e mudanças de permissão	Tokens/API keys completos
Erros 4xx e 5xx com contexto	Dados de cartão de crédito (PCI)
IPs e User-Agents	Dados pessoais sensíveis (CPF, documentos)
Mudanças em dados críticos (audit trail)	Dados de saúde (HIPAA/LGPD)

17.2 Structured Logging

```
const pino = require("pino");
const logger = pino({ level: process.env.LOG_LEVEL || "info" });

// Log de segurança
logger.warn({
  event: "login_failed",
  email: req.body.email,
  ip: req.ip,
  userAgent: req.headers["user-agent"],
  reason: "invalid_password",
  timestamp: new Date().toISOString(),
});
```

17.3 Alertas Críticos

- Mais de 10 login failures por IP em 5 minutos
- Tentativa de acesso a endpoint admin por usuário não-admin
- Spike de 4xx/5xx errors (possível ataque ou bug)
- Queries incomuns no banco (possível SQLi)
- Novo IP acessando conta admin (possível comprometimento)

17.4 Incident Response Plan

5. DETECTAR: Alertas automáticos via SIEM/Datadog/Grafana

6. CONTER: Isolar sistema afetado, revogar tokens comprometidos
7. ERRADICAR: Identificar e corrigir a vulnerabilidade
8. RECUPERAR: Restaurar serviço, verificar integridade dos dados
9. DOCUMENTAR: Post-mortem sem culpabilização, melhorar defesas
10. NOTIFICAR: LGPD exige notificação à ANPD em até 2 dias úteis

18. Checklist Final de Segurança

Use esta checklist como referência para auditar qualquer serviço web ou SaaS. Cada item deve ser verificado antes de ir para produção e revisado periodicamente.

Autenticação & Sessão

- Senhas com hash bcrypt (cost 12+) ou Argon2id
- MFA disponível para todos os usuários
- Rate limiting no login (5 tentativas / 15min)
- JWT com expiração curta (15min access, 7d refresh)
- Cookies httpOnly, Secure, SameSite=Strict
- Logout invalida token no backend

Autorização

- Verificação server-side em CADA endpoint
- RBAC ou ABAC implementado
- Multi-tenancy com isolamento (RLS ou filtro obrigatório)
- IDs não-sequenciais (UUIDs v4)

Input & Output

- Validação de schema (Zod/Joi) em todas as rotas
- Prepared statements / ORM para queries
- Output encoding / sanitização HTML
- CSP header configurado
- Respostas de API com projection (sem dados extras)

Infraestrutura

- HTTPS obrigatório + HSTS
- Headers de segurança (Helmet.js ou equivalente)
- CORS restrito a domínios confiáveis
- Containers não rodam como root
- Secrets em serviço de gerenciamento, não no código
- Imagens Docker escaneadas

Operações

- npm audit / dependabot habilitado

- SAST no CI/CD (Semgrep, SonarQube)
- Structured logging sem dados sensíveis
- Alertas para eventos de segurança
- Plano de incident response documentado
- Backups criptografados e testados
- Pentest periódico (mínimo anual)

19. Referências e Recursos

Guias e Frameworks

- OWASP Top 10: https://owasp.org/Top10/
- OWASP API Security Top 10: https://owasp.org/API-Security/
- OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org/
- CWE (Common Weakness Enumeration): https://cwe.mitre.org/
- NIST Cybersecurity Framework: https://nist.gov/cyberframework

Ferramentas Recomendadas

Categoria	Ferramentas
SAST (código)	Semgrep, SonarQube, CodeQL
DAST (runtime)	OWASP ZAP, Burp Suite, Nuclei
Dependências	Dependabot, Snyk, npm audit
Containers	Trivy, Snyk Container, Grype
Secrets	TruffleHog, GitLeaks, detect-secrets
WAF	Cloudflare WAF, AWS WAF, ModSecurity
Monitoramento	Datadog, Grafana + Loki, ELK Stack
Headers	securityheaders.com, Mozilla Observatory

Compliance & Regulamentação

- LGPD (Brasil): Lei 13.709/2018 — Proteção de dados pessoais
- SOC 2 Type II: Controles de segurança para SaaS
- ISO 27001: Sistema de gestão de segurança da informação
- PCI DSS: Para processamento de pagamentos

Este documento é um guia vivo e deve ser atualizado conforme novas vulnerabilidades são descobertas e novas defesas são desenvolvidas. A segurança não é um destino, é um processo contínuo.